# MECHATRONICS AND IOT

# ASSIGNMENT 7

# Design and develop a Water Level Detection and Flood Alert System using Ultrasonic Sensor.


**TEAM MEMBERS:**

**KOWSHIK K 24MER025**

**NITHISH J 24MER037**

**KAVIN KRISHNA D S 24MER024**

**LOGESH S 24MER027**

# 1. Project Overview

The **Water Level Detection and Flood Alert System** is an IoT-based monitoring system designed to measure the water level in **tanks, rivers, reservoirs, and small dam models**. The system uses an **ultrasonic sensor** to measure the distance between the sensor and the water surface without making direct contact with the water.

An **ESP32** acts as the main controller. It receives the distance measured by the ultrasonic sensor, calculates the actual water level, and compares it with predefined threshold levels. When the water level reaches a warning or danger level, the system activates a **buzzer and LED** to provide a local warning. The ESP32 can also send the water-level information through Wi-Fi to an IoT dashboard for remote monitoring.

# 2. Problem Statement

Water overflow is a common problem in **rivers, water tanks, reservoirs, dams, and drainage systems**, especially during heavy rainfall, sudden water inflow, or improper water management. When the water level rises beyond the safe limit, it can result in flooding and cause damage to **agricultural land, roads, buildings, electrical systems, industrial facilities, and other infrastructure**. In some situations, people living near water bodies may not receive sufficient warning before the water reaches a dangerous level.

Traditional water-level monitoring methods often depend on **manual inspection or periodic measurement**. A person may need to physically visit the location and check the water level. This method is time-consuming and may be unsafe during heavy rainfall, storms, or rapidly rising water conditions. Since the water level is not monitored continuously, a sudden increase may remain unnoticed until it reaches a critical level.

The major problems associated with conventional water-level monitoring are:

- **Manual measurement:** Water levels may need to be checked manually, which requires human effort and regular inspection.
- **Lack of continuous monitoring:** Traditional methods may not provide real-time information about changes in water level.

- **Delayed detection:** A sudden rise in water level may not be identified early enough.
- **Delayed warning:** Without an automatic alert system, nearby people may not receive timely overflow warnings.
- **Difficult remote access:** Rivers, dams, reservoirs, and other water bodies may be located in remote or difficult-to-access areas.

## 3. Proposed Solution

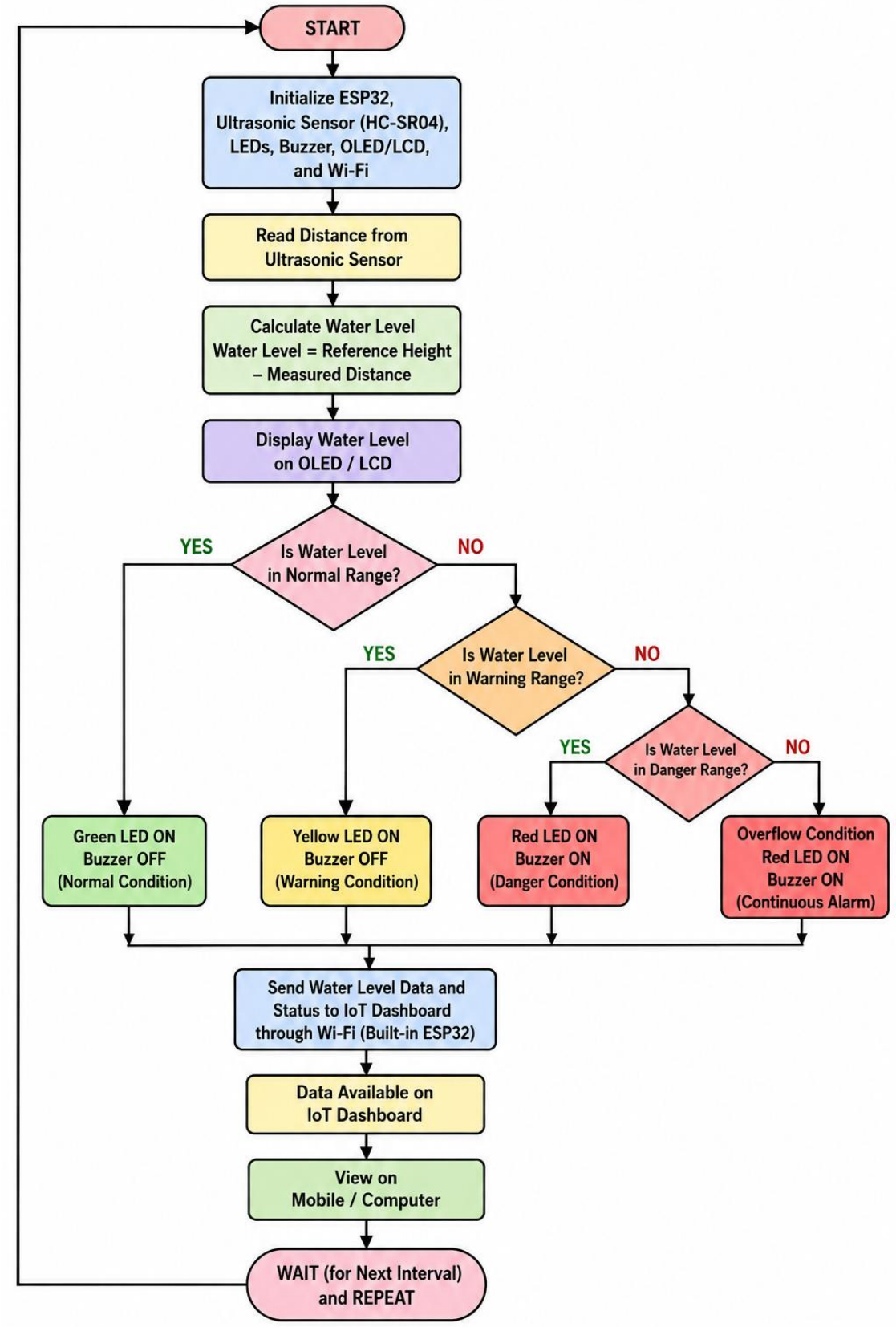
- ✓ The proposed **Water Level Detection and Flood Alert System** uses an **ultrasonic sensor** mounted above the water surface to continuously measure the water level. The ultrasonic sensor works by transmitting a high-frequency sound pulse toward the water surface. The pulse is reflected back when it reaches the water, and the sensor measures the time taken for the echo to return.
- ✓ An **ESP32 microcontroller** receives this time information and calculates the distance between the ultrasonic sensor and the water surface. Since the sensor is installed at a fixed height, the actual water level can be calculated using the reference height.
- ✓ **Water Level = Reference Height − Measured Distance**
- ✓ For example, if the distance from the sensor to the bottom of the tank is **100 cm** and the ultrasonic sensor measures **30 cm** between the sensor and water surface:
- ✓ **Water Level = 100 − 30 = 70 cm**
- ✓ The calculated water level is continuously compared with predefined threshold values. Based on the water level, the system provides different warning indications using **LEDs and a buzzer**.

The ESP32 can also transmit the measured water-level data through **Wi-Fi** to an IoT platform such as **Adafruit IO**. This allows the water level and warning status to be monitored remotely using a mobile phone or computer.

## Main Features

- Continuous water-level measurement.
- Non-contact measurement using an ultrasonic sensor.
- Automatic calculation of water level.
- Multiple warning levels for better safety.
- LED indication for normal, warning, and danger conditions.
- Buzzer alert during critical water levels.

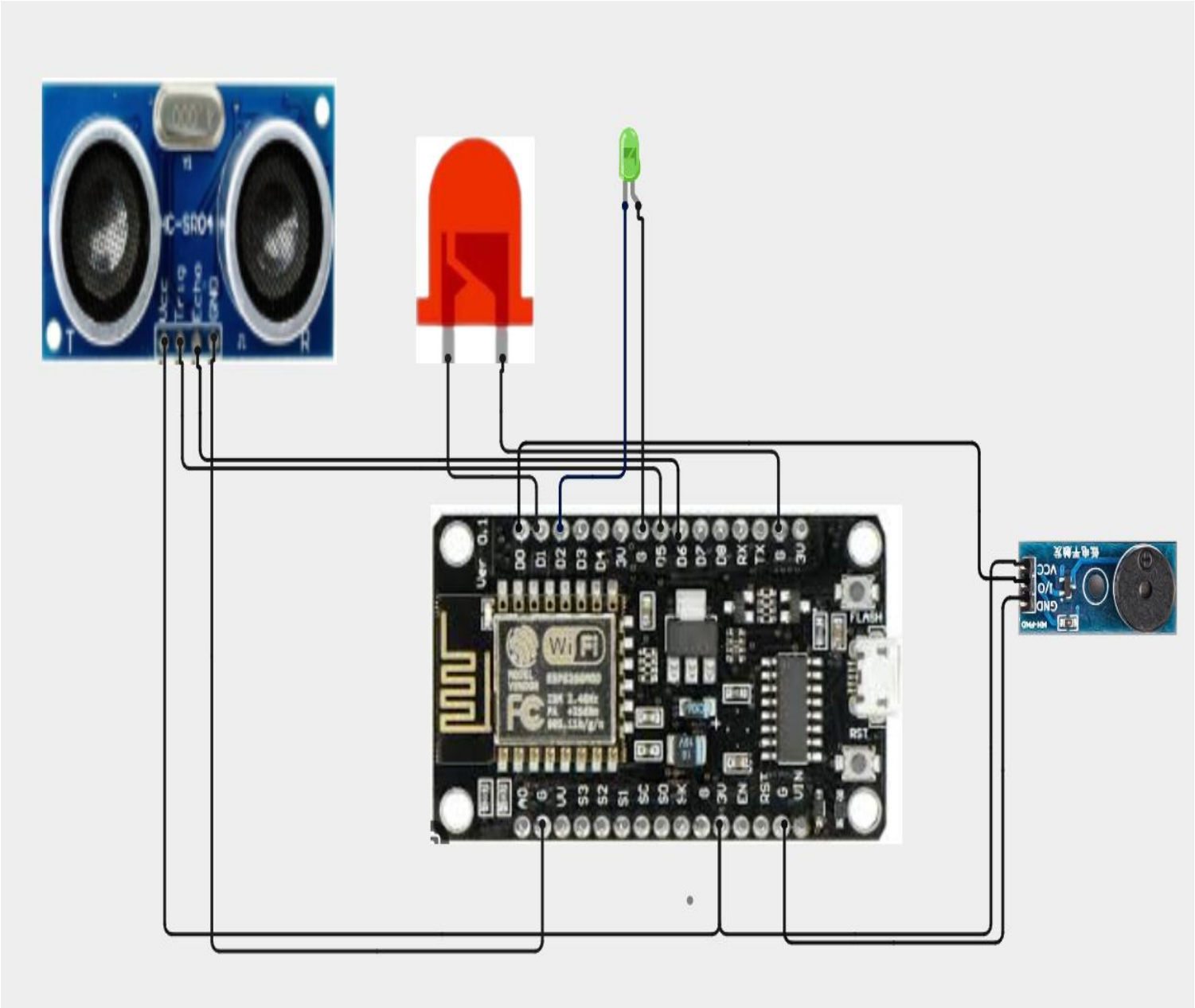
# 4. System Architecture



# 5. Circuit Table

| S.No | Component | Pin | ESP32 Connection | Function |
|---|---|---|---|---|
| 1 | HC-SR04 | VCC | 5V / VIN | Sensor power |
| 2 | HC-SR04 | GND | GND | Ground |
| 3 | HC-SR04 | TRIG | GPIO 5 | Sends ultrasonic pulse |
| 4 | HC-SR04 | ECHO | GPIO 18* | Receives reflected pulse |
| 5 | Green LED | Anode (+) | GPIO 25 | Normal level indication |
| 6 | Green LED | Cathode (−) | GND | Ground |
| 7 | Yellow LED | Anode (+) | GPIO 26 | Warning level indication |
| 8 | Yellow LED | Cathode (−) | GND | Ground |
| 9 | Red LED | Anode (+) | GPIO 27 | Danger level indication |
| 10 | Red LED | Cathode (−) | GND | Ground |
| 11 | Buzzer | + | GPIO 14 | Flood warning |
| 12 | Buzzer | − | GND | Ground |
| 13 | All Components | GND | GND | Common ground |

# 6. Circuit Design



# 7. Algorithm

## Step-by-Step Algorithm

- **Start the system**
  Switch ON the power supply to the ESP32, ultrasonic sensor, LEDs, buzzer, and other connected components.
- **Initialize the ESP32**
  Configure the required GPIO pins for the HC-SR04 ultrasonic sensor, green LED, yellow LED, red LED, and buzzer.

- **Initialize the ultrasonic sensor**
  Configure the TRIG pin as an output and the ECHO pin as an input. The sensor is used to measure the distance between the sensor and the water surface.
- **Set the reference height**
  Enter the fixed distance between the ultrasonic sensor position and the bottom of the tank or water-level model. This value is used to calculate the actual water level.
- **Trigger the ultrasonic sensor**
  The ESP32 sends a short trigger signal to the HC-SR04 sensor to start the distance measurement.
- **Transmit the ultrasonic pulse**
  The HC-SR04 sends an ultrasonic pulse toward the surface of the water.
- **Receive the reflected pulse**
  The ultrasonic wave reflects from the water surface and returns to the sensor. The ECHO pin provides the time taken for the pulse to return.
- **Calculate the distance**
  The ESP32 calculates the distance between the sensor and the water surface using the measured echo time.
- **Calculate the water level**
  The actual water level is calculated using:
  - **Water Level = Reference Height − Measured Distance**
- **Check the measured value**
  The calculated water level is checked to ensure that it is within the expected measurement range.
- **Compare with predefined thresholds**
  The ESP32 compares the water level with the programmed normal, warning, danger, and overflow limits.
- **Normal condition**
  If the water level is within the normal range, the **green LED is turned ON** and the buzzer remains OFF.
- **Warning condition**
  If the water level reaches the warning range, the **yellow LED is turned ON** to indicate that the water level is increasing and should be monitored carefully.

# 8. Coding

```
#include <ESP8266WiFi.h>
#include "AdafruitIO_WiFi.h"

// ============================================================
// WiFi Credentials
// ============================================================

#define WIFI_SSID "Luky003"
#define WIFI_PASS "kowshik03"

// ============================================================
// Adafruit IO Credentials
// ============================================================

#define IO_USERNAME "Nithish_037"
#define IO_KEY "aio_fDCY0249PAv0qGua49svZIm8oV9D"

AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);

// ============================================================
// Adafruit IO Feeds
// ============================================================

AdafruitIO_Feed *waterLevelFeed = io.feed("water-level");
AdafruitIO_Feed *distanceFeed   = io.feed("distance");
AdafruitIO_Feed *statusFeed     = io.feed("status");

// ============================================================
```

```
// NodeMCU ESP8266 Pin Connections
// ========================================================

#define TRIG_PIN   14   // D5
#define ECHO_PIN   12   // D6

#define GREEN_LED   5   // D1
#define YELLOW_LED  4   // D2
#define RED_LED    13   // D7

#define BUZZER     15   // D8

// ========================================================
// Tank Height
// ========================================================

const float tankHeight = 100.0;   // Tank height in cm

// ========================================================
// Variables
// ========================================================

bool alertSent = false;
unsigned long lastUpload = 0;

// ========================================================
// SETUP
// ========================================================
```

```
void setup() {

  Serial.begin(115200);

  // Ultrasonic sensor
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);

  // LEDs
  pinMode(GREEN_LED, OUTPUT);
  pinMode(YELLOW_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);

  // Buzzer
  pinMode(BUZZER, OUTPUT);

  // Initially OFF
  digitalWrite(TRIG_PIN, LOW);

  digitalWrite(GREEN_LED, LOW);
  digitalWrite(YELLOW_LED, LOW);
  digitalWrite(RED_LED, LOW);
  digitalWrite(BUZZER, LOW);

  // ====================================================
  // Connect to Adafruit IO
  // ====================================================

  Serial.println();
```

```
  Serial.println("================================");
  Serial.println(" WATER LEVEL FLOOD ALERT SYSTEM");
  Serial.println("================================");
  Serial.println("Connecting to Adafruit IO...");

  io.connect();

  while (io.status() < AIO_CONNECTED) {
    Serial.print(".");
    delay(500);
  }

  Serial.println();
  Serial.println("Adafruit IO Connected!");
  Serial.println("System Started");
  Serial.println("--------------------------------");
}

// ============================================================
// LOOP
// ============================================================

void loop() {

  // Keep Adafruit IO connection alive
  io.run();

  // ============================================================
  // Read ultrasonic distance
```

```
// ========================================================

float distance = readDistance();

// ========================================================
// Calculate water level percentage
// ========================================================

float level = ((tankHeight - distance) / tankHeight) * 100.0;

// Keep level between 0 and 100
level = constrain(level, 0, 100);

// ========================================================
// Status variable
// ========================================================

String status = "SAFE";

// ========================================================
// SAFE: Below 40%
// ========================================================

if (level < 40) {

  digitalWrite(GREEN_LED, HIGH);
  digitalWrite(YELLOW_LED, LOW);
  digitalWrite(RED_LED, LOW);
  digitalWrite(BUZZER, LOW);
```

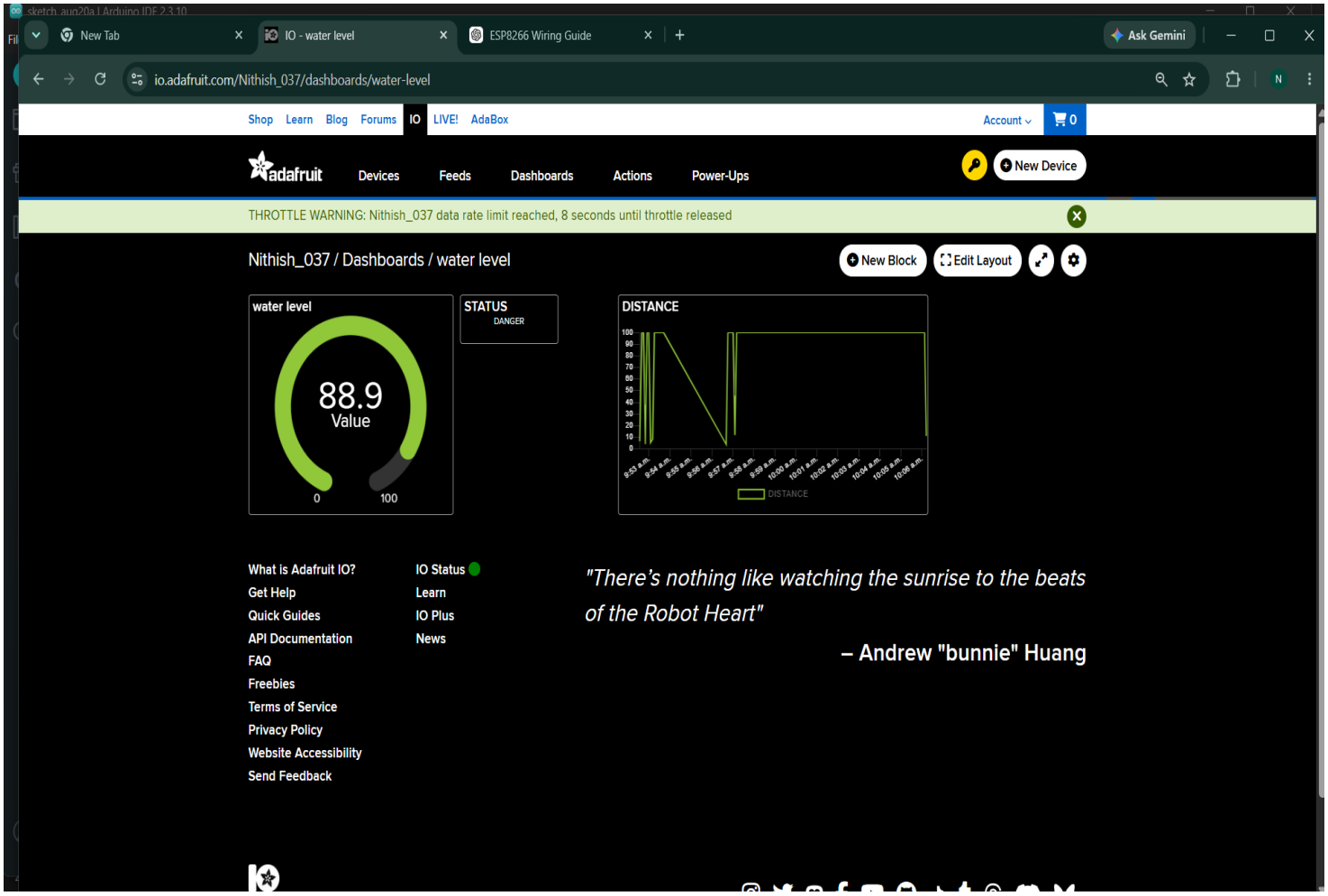
```
  status = "SAFE";

  // Reset alert when water level becomes safe
  alertSent = false;
}

// ========================================================
// WARNING: 40% to below 80%
// ========================================================

else if (level < 80) {

  digitalWrite(GREEN_LED, LOW);
  digitalWrite(YELLOW_LED, HIGH);
  digitalWrite(RED_LED, LOW);
  digitalWrite(BUZZER, LOW);

  status = "WARNING";

  // Reset alert
  alertSent = false;
}

// ========================================================
// DANGER: 80% and above
// ========================================================

else {
```

```
  digitalWrite(GREEN_LED, LOW);
  digitalWrite(YELLOW_LED, LOW);
  digitalWrite(RED_LED, HIGH);
  digitalWrite(BUZZER, HIGH);

  status = "DANGER";

  // Print alert only once
  if (!alertSent) {

    Serial.println();
    Serial.println("!!! DANGER ALERT !!!");
    Serial.println("Water level reached 80% or above!");
    Serial.println("SMS Sent: Danger Alert");

    alertSent = true;
  }
}

// ========================================================
// SERIAL MONITOR
// ========================================================

Serial.print("Distance: ");
Serial.print(distance, 1);
Serial.print(" cm");

Serial.print(" | Water Level: ");
```

```
  Serial.print(level, 1);
  Serial.print("%");

  Serial.print(" | Status: ");
  Serial.println(status);

  // =======================================================
  // SEND DATA TO ADAFRUIT IO EVERY 5 SECONDS
  // =======================================================

  if (millis() - lastUpload >= 5000) {

    waterLevelFeed->save(level);
    distanceFeed->save(distance);
    statusFeed->save(status);

    Serial.println("Data uploaded to Adafruit IO");

    lastUpload = millis();
  }

  // Small delay
  delay(5000);
}

// =========================================================
// ULTRASONIC SENSOR FUNCTION
// =========================================================
```

```
float readDistance() {
  // Send trigger pulse
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
  // Read echo
  long duration = pulseIn(ECHO_PIN, HIGH, 30000);
  // If no echo received

  if (duration == 0) {
    return tankHeight;
  }
  // Convert time to distance in cm
  float distance = duration * 0.0343 / 2.0;
  // Prevent impossible values

  if (distance > tankHeight) {
    distance = tankHeight;

  }
  if (distance < 0) {
    distance = 0;
  }

  return distance;
}
```

# 9. System Working




The ultrasonic sensor is placed above the water surface. The ESP32 periodically triggers the sensor, and the sensor sends an ultrasonic pulse toward the water.

When the pulse reaches the water surface, it is reflected back toward the sensor. The sensor calculates the travel time, and the ESP32 converts this time into distance.

As the water level rises:

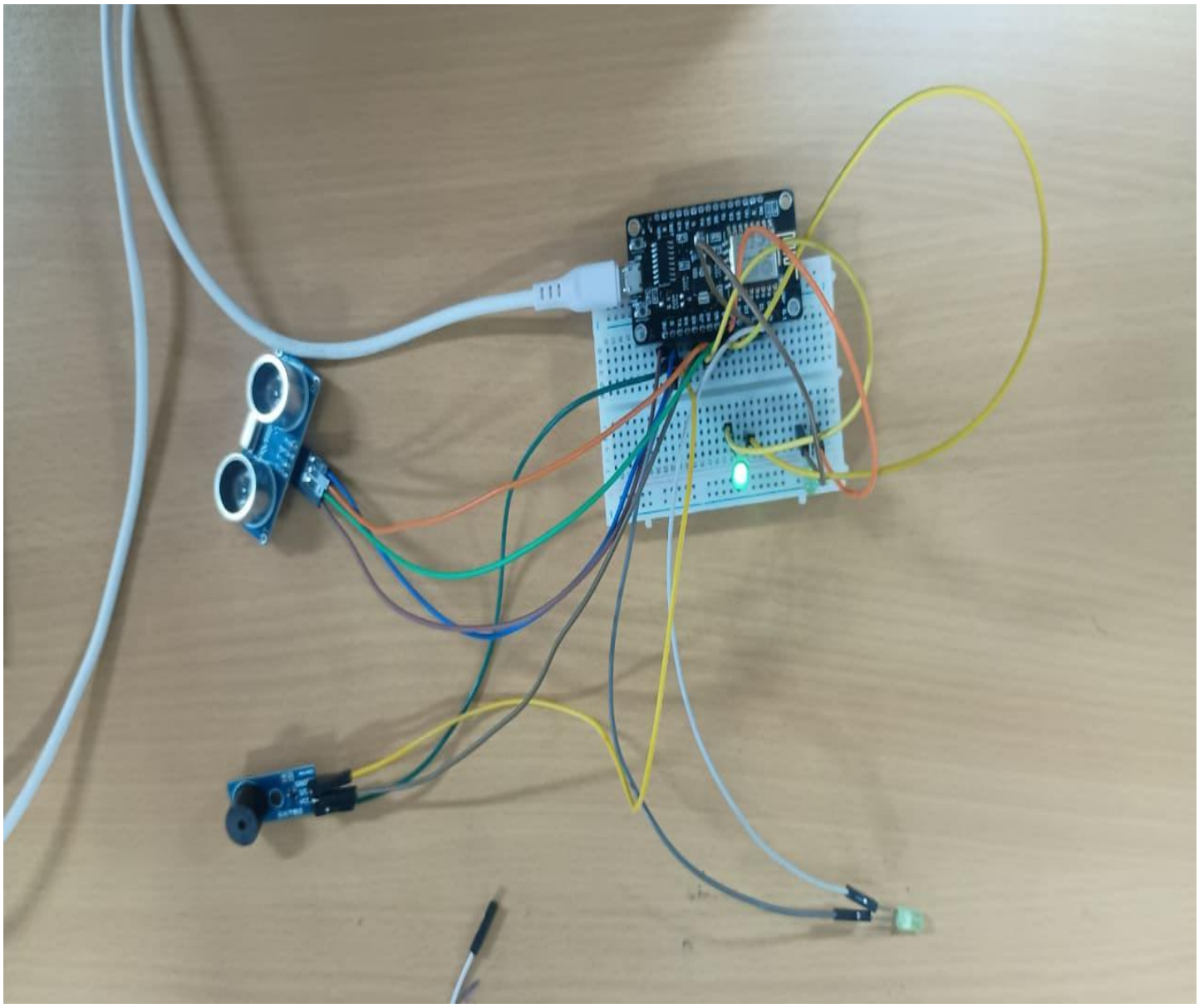
**Distance measured by sensor decreases → Calculated water level increases**

The ESP32 compares the calculated water level with the programmed thresholds.

# 10. Bill of Materials

| S.No | Component | Quantity | Approx. Cost |
|---|---|---|---|
| 1 | ESP32 Development Board | 1 | ₹350–500 |
| 2 | HC-SR04 Ultrasonic Sensor | 1 | ₹50–100 |
| 3 | Green LED | 1 | ₹5 |
| 4 | Yellow LED | 1 | ₹5 |
| 5 | Red LED | 1 | ₹5 |
| 6 | Buzzer | 1 | ₹10–20 |
| 7 | 220 Ω Resistors | 3 | ₹5–10 |
| 8 | Resistors for ECHO voltage divider | 2 | ₹5–10 |
| 9 | Breadboard | 1 | ₹50–100 |
| 10 | Jumper Wires | 1 set | ₹50–100 |
| 11 | USB Cable | 1 | ₹50–100 |
| 12 | Water Tank/Container for Prototype | 1 | ₹50–150 |

# 11. Prototype Implementation



## Implementation steps

### Step 1 – Prepare the tank

Use a transparent container so that the water level can be easily observed.

### Step 2 – Mount the ultrasonic sensor

Fix the HC-SR04 at the top of the tank, facing vertically downward.

### Step 3 – Connect the ESP32

Connect the ultrasonic sensor, LEDs, and buzzer according to the circuit table.

**Step 4 – Program the ESP32**

Upload the Arduino code and set the correct tank height.

**Step 5 – Test different water levels**

Gradually add water to the tank and observe the sensor readings.

**Step 6 – Test the alarm**

When the water reaches the warning/danger level, verify that the corresponding LED and buzzer operate correctly.

**Step 7 – IoT integration**

Connect the ESP32 to Wi-Fi and send the water-level data to an IoT dashboard for remote monitoring.

## 12. Result and Performance Analysis

The **Water Level Detection and Flood Alert System** provides continuous and non-contact measurement of water level using an ultrasonic sensor. The ESP32 accurately calculates the water level from the measured distance and compares it with predefined threshold values. When the water level reaches the warning or danger range, the LED and buzzer are activated immediately.

## Performance Points

- **Water Level Detection:** Continuously measures the water level in real time.
- **Measurement Accuracy:** Provides reasonably accurate readings for a small prototype when the ultrasonic sensor is correctly positioned.
- **Response Time:** The system responds quickly when the water level crosses the programmed threshold.
- **Alert Performance:** Buzzer and LEDs provide clear warning indications for rising water levels.
- **Non-Contact Measurement:** The sensor measures the water level without touching the water, reducing sensor contamination.

## Advantages

- **Non-contact measurement** – The ultrasonic sensor does not need to touch the water.
- **Real-time monitoring** – Water level can be monitored continuously.
- **Early warning** – Alerts can be generated before the water reaches the overflow level.
- **Wireless monitoring** – ESP32 can transmit data through Wi-Fi.
- **Low-cost prototype** – The system can be developed using inexpensive components.
- **Easy installation** – The sensor can be mounted above a tank or water body.
- **Automatic operation** – Continuous manual measurement is not required.
- **IoT compatible** – Water-level data can be displayed remotely.
- **Expandable** – Additional rain, flow, or water-quality sensors can be added.

## Limitations

- **Ultrasonic measurement can be affected by water-surface movement**, especially in turbulent water.
- **Heavy rain, fog, or obstructions** may affect ultrasonic measurements.
- **HC-SR04 has limited range**, so it is mainly suitable for prototype and small-scale applications.
- **Wi-Fi is required** for remote IoT monitoring.
- **Power failure** can stop the monitoring system unless backup power is provided.
- **Sensor positioning is important** for reliable readings.
- **A single sensor may not be sufficient for a real dam or river flood-warning system.**